Dataset Description:

The dataset consists of **290 RGB satellite images** with their corresponding binary segmentation masks indicating flooded areas.

Raw Data Properties:

- Image shape: `(551, 893, 3)`
- Mask shape: `(551, 893)`
- Masks are single-channel grayscale images.

Dataset Class Design

A custom PyTorch dataset class `READ_FLOOD_AREA_DATA_SET` was implemented to handle:

- Image loading / preprocessing
- Target loading / preprocessing
- Creating the Mask using 2 methods
- Optional augmentation
- Multiple operational modes (train / evaluation / test)

This modular design allows controlled experimentation with partial supervision strategies.

Image Processing Pipeline

Loaded in **RGB format**, normalized for stable training, resized to a fixed resolution to reduce computational cost and ensure consistent input dimension
The downscaling is made separately where the image is interpolated using BICUBIC to preserve the details and sharper edges, and mask is interpolated using Nearest-exact to guarantee the closest match
Interpolate the decreased image using bicubic method to produce smoother downsampled images and preserve spatial details, besides using antialias to reduce high frequency artifacts when downsampling.
Finally normalize the image pixels in each channel to be in same range as the original pretrained images were trained for the encoder using imagenet dataset

Target Processing Pipeline

Loaded in **gray scale format as a single channel image**, resized to the same image fixed resolution but using a different Interpolation method.
Interpolated using NEAREST_EXACT method to prevent label interpolation, avoid fractional pixel values and preserve binary class integrity, then convert target pixels into 2 segments 0 for background, and 1 for flood, target shape (H,W)

Augmentation:

This mode is optional and better to be used to generate pairs of image target data to increase the dataset size in to save the created data locally to be used in the training beside the original data, the current approach is to use the on the fly method
Augmentation is split into two ways:
1. Photometric for image only to apply random brightness and random contrast.
2.Geometric for image and target to apply random horizontal flip
The geometric transforms are applied jointly to image and mask to preserve spatial alignment.

Mask Sampling:

To simulate partial supervision, a binary mask is generated to hide pixel values from the calculation in the loss function, where 1 → pixel contributes to loss and 0 → pixel ignored using 2 strategies.

1. Random Sampling:

Randomly drops a percentage of pixels from the full mask, and it's useful for studying robustness under sparse supervision.

2. Class Balanced Sampling:

Drops pixels separately from water class and background class, which is useful for preventing dominance of background class

---

Operations:

1. Train mode → image, target, mask
2. Evaluation mode → image, target
3. Test mode → image

Model Evaluation Strategy

Flood segmentation datasets often contain images with highly varying flood areas:

- Some images contain large flooded regions
- Others contain only a few flood pixels

To avoid bias evaluation, and ensure a fair and comprehensive assessment:

1. **Global Pixel-Wise Evaluation**
2. **Per Image Spatial Evaluation**

Global Pixel Wise Evaluation

This approach treats each pixel equally, regardless of which image it belongs to, It evaluates the model as a pixel-level binary classifier over the entire validation set.

This would reflect overall segmentation performance, suitable when viewing the task as large scale pixel classification and give strong signal about class imbalance behavior

All True Positives (TP), False Positives (FP), and False Negatives (FN) are aggregated across all pixels and all images

Then the following metrics are computed globally: **Precision, Recall and F1 Score**

Per Image Spatial Evaluation

This approach used to solve bias cases where images with large flood areas dominate or images with small flood regions barely influence the result

Using spatial metrics are computed per image, so for each image compute IOU and Dice then take the mean across all images

Intersection over union (IOU) and Dice are spatial metrics, they are helpful because they measure region overlap, evaluate segmentation quality and are sensitive to boundary alignment

Using both Strategies ensures the following:

- Fair evaluation across dataset imbalance
- Robust understanding of segmentation behavior
- Avoiding bias toward large flood regions

Training Strategy

Due to the relatively small dataset size (290 images), a K-Fold Cross Validation strategy was adopted to ensure robust and unbiased model evaluation and more model generalization
Number of folds: K = 5, each fold: 80% used for training and 20% used for validation

---

Model Architecture

The segmentation model is: **U-Net with Xception encoder (ImageNet pretrained)**

→ Encoder weights initialized from ImageNet
→ Output channel: 1 (binary segmentation)
→ Sigmoid activation applied during evaluation

Spatial Focal Loss:

To solve the main challenge of segmentation problem which is class imbalance as background pixels dominate most images, and lack of accrued labeled pixels so, will use focal loss + mask

During training, only a subset of pixels contribute to the loss (using masks).
Why not a standard Binary Cross Entropy?
Because it will treat easy and hard examples equally, and the loss would be dominated by background pixels, so focal loss would give less weights to easy examples classification (background) to influence the gradients values, and using the mask would adapt the network to deal with lack in segmented/labeled pixels

Focal Loss Overview:

The idea is to down weight the influence of easy examples/ common class ex background pixels, that give the rare/hard examples ex flood pixels the chance to have more influence to the loss value
Same BCE equation with extra 2 hyper parameters

- $\alpha \in [0,1]$ → is a weight for each class, where the rare class is multiplied by a higher weight (α), and the common class is multiplied by a lower weight ($1-\alpha$), this increases the loss contribution of the rare class and decreases the contribution of the dominant class, so the model pays more attention to the rare class during training.

- Gamma → responsible for down weighting common/ easy examples to reduces the influence of easy examples on the loss function, resulting in more attention being paid to hard examples

To simulate sparse labeling, a binary supervision mask is applied, 1 → pixel contributes to loss and 0 → pixel ignored

---

Optimization

Optimizer: Adam
Gradients monitored for NaN / Inf for numerical stability

Training Loop

For each fold:

1. Train the model on kaggle, GPU P100
2. Initialize model
3. Train for num_epoch
4. Compute final validation metrics on held-out fold

5. Store metrics
6. Repeat for next fold

Final Reporting

For each drop percentage experiment, the following are reported:
→ Precision (global)
→ Recall (global)
→ F1 Score (global)
→ IoU (per image mean)
→ Dice (per image mean)

Final performance across all folds is reported as: Metric: Mean ± Std

---

Experiments configurations:

Image size: 224,224
Number of epochs: 10
Batch size: 16
Threshold: 0.5
Mask strategy: Random
No data augmentation is applied
Alpha value for focal loss:0.25
Gamma value for focal loss: for the first experiment 2, and for the second 5

Experiment 1 – Random Sampling – Loss Gamma: 2, Loss Alpha: 0.25

Drop %	Precision	Recall	F1 Score	IoU	Dice
0.1	0.9310 ± 0.0159	0.7793 ± 0.0414	0.8478 ± 0.0265	0.6971 ± 0.0480	0.8035 ± 0.0426
0.3	0.9334 ± 0.0209	0.7723 ± 0.0264	0.8449 ± 0.0179	0.6909 ± 0.0333	0.7983 ± 0.0284
0.5	0.9392 ± 0.0035	0.7482 ± 0.0614	0.8314 ± 0.0372	0.6685 ± 0.0676	0.7779 ± 0.0585

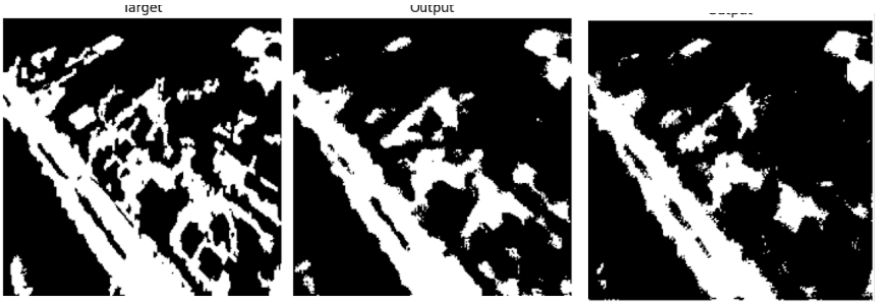
0.7	0.9365 ± 0.0136	0.7613 ± 0.0485	0.8390 ± 0.0301	0.6841 ± 0.0568	0.7914 ± 0.0473

Experiment 1 Conclusion:

Increasing drop percentage generally
→ Slightly increases precision
→ Decreases recall
→ Reduces spatial metrics (IoU, Dice)
This suggests:
→ High drop acts as regularization
→ Excessive pixel dropping reduces spatial learning consistency.

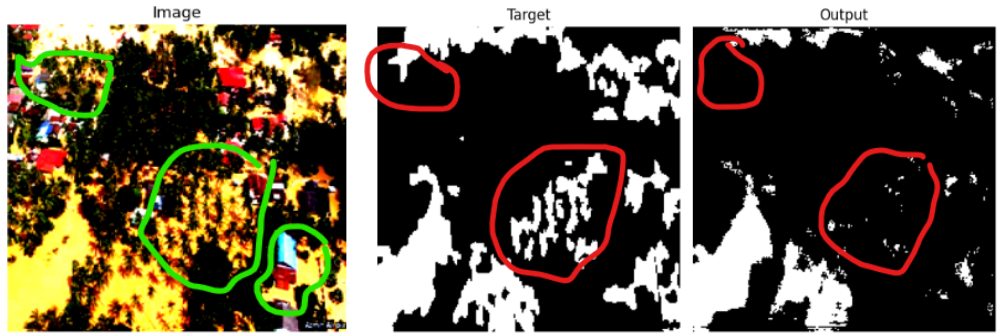
Although drop=0.7 achieves slightly better quantitative metrics (IoU, Dice, F1), visual inspection suggests smoother and less sharp boundaries compared to drop=0.5. This indicates that stronger pixel dropping acts as regularization, improving region coverage while slightly reducing boundary precision.

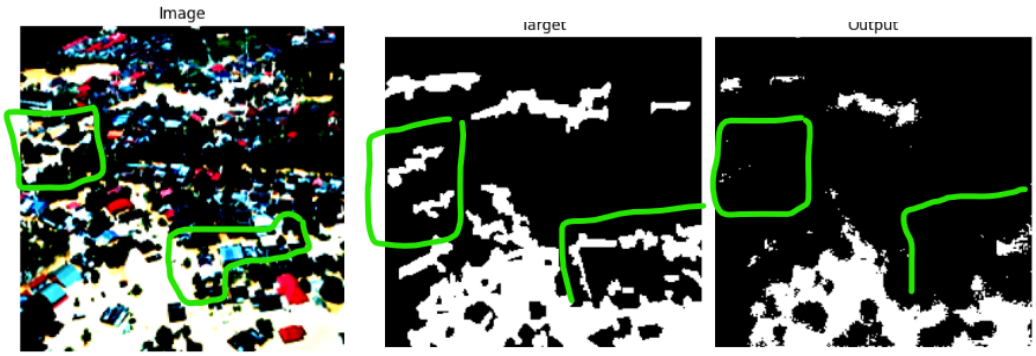
Drop 0.5 vs Drop 0.7 at fold 4



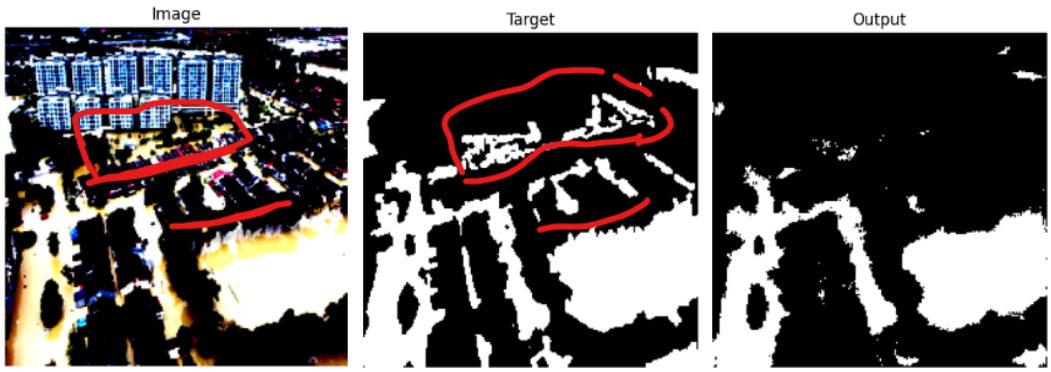
For higher drop ratios (0.5 and 0.7), the model successfully captures large flood regions. However, it tends to misclassify small flood areas located between objects such as buildings or narrow streets between dense trees.

Examples from drop = 0.7



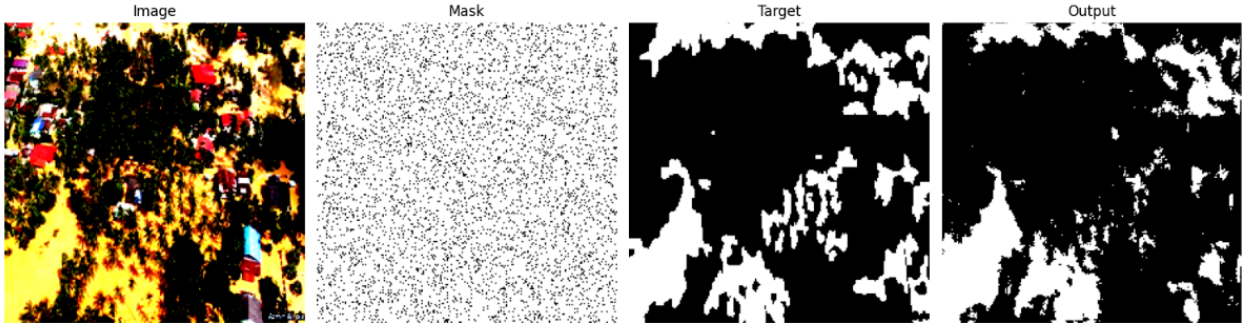


Examples from drop=0.5

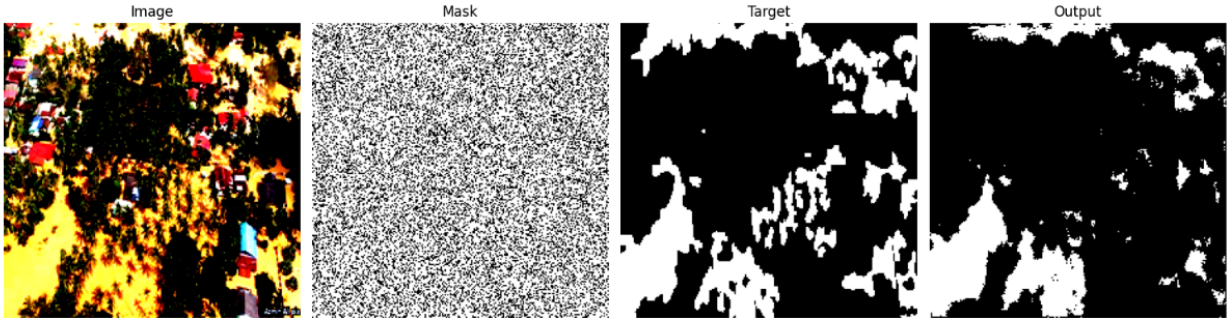


Examples from validation on each drop for experiment 1:

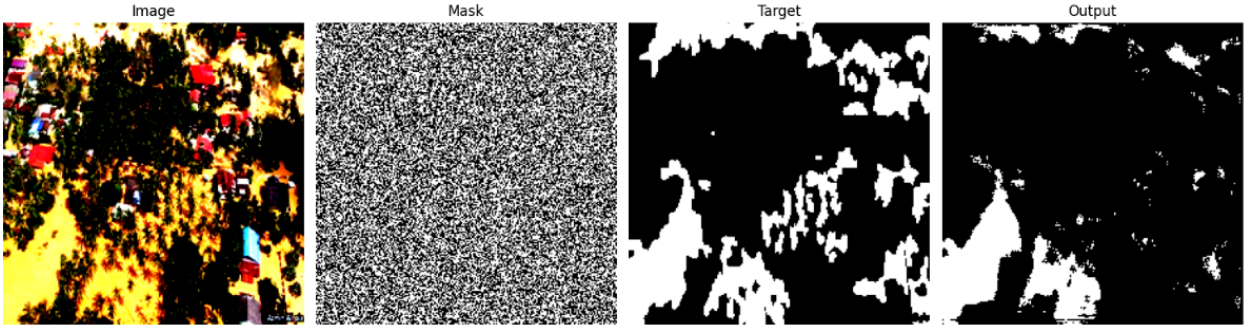
Drop: 0.1



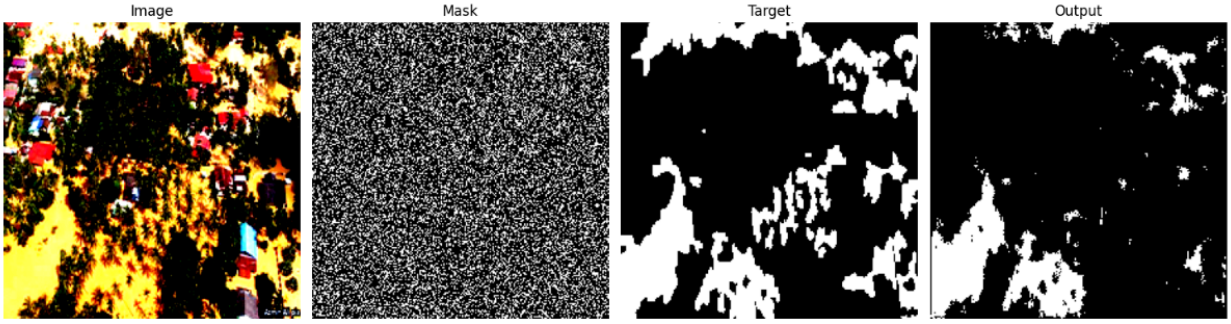
Drop: 0.3



Drop: 0.5



Drop 0.7

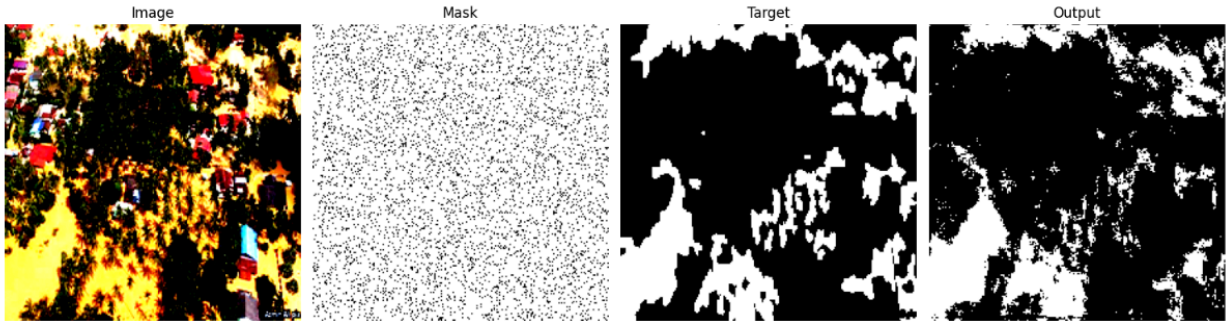


Experiment 2 – Random Sampling – Loss Gamma: 5, Loss Alpha: 0.25

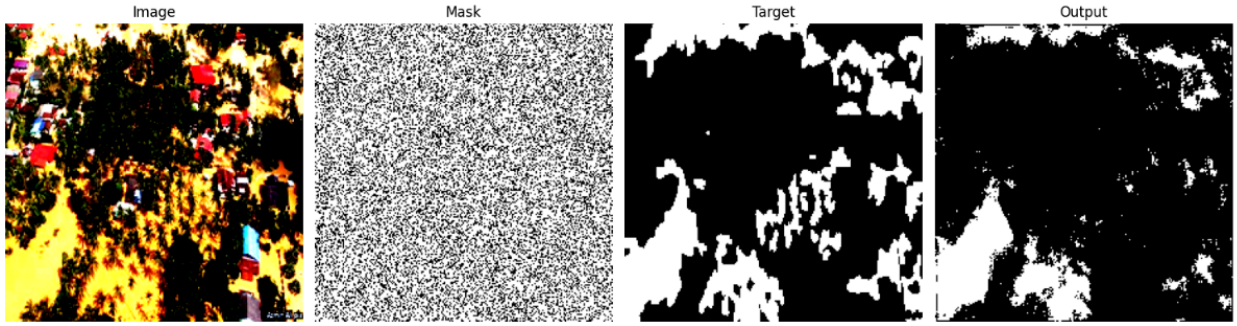
Drop %	Precision	Recall	F1 Score	IoU	Dice
0.1	0.9134 ± 0.0228	0.7528 ± 0.0370	0.8250 ± 0.0266	0.6617 ± 0.0442	0.7772 ± 0.0393
0.3	0.9262 ± 0.0135	0.7339 ± 0.0390	0.8183 ± 0.0238	0.6487 ± 0.0434	0.7653 ± 0.0391
0.5	0.9222 ± 0.0164	0.7021 ± 0.0438	0.7965 ± 0.0297	0.6121 ± 0.0516	0.7334 ± 0.0465
0.7	0.9208 ± 0.0202	0.7279 ± 0.0321	0.8127 ± 0.0246	0.6395 ± 0.0468	0.7591 ± 0.0409

Examples from validation on each drop for experiment 2:

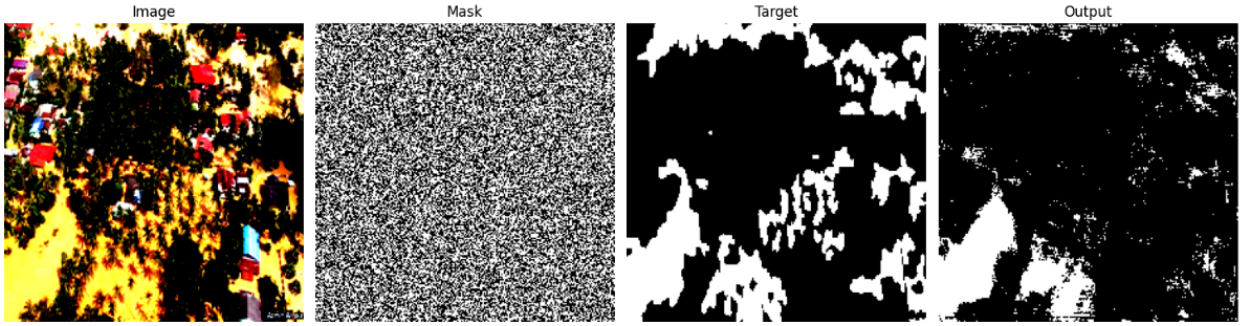
Drop: 0.1



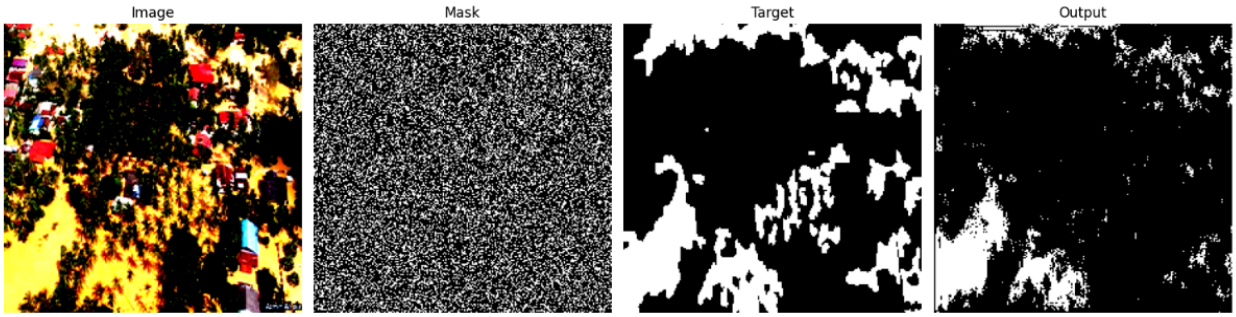
Drop: 0.3



Drop: 0.5



Drop: 0.7



Observations from experiment 1, and experiment 2

→ Increasing gamma from 2 → 5 resulted in:

- Lower Recall across all drop values
- Lower IoU and Dice
- Lower F1 Score

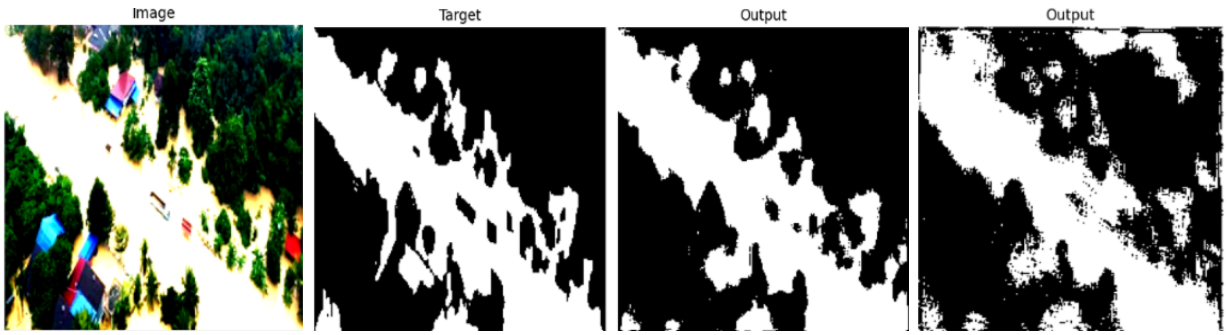
Increasing gamma to 5 resulted in a consistent decrease across all evaluation metrics compared to gamma=2. Although focal loss with higher gamma emphasizes hard-to-classify pixels, the model became overly focused on difficult samples, reducing its overall segmentation performance. Given the relatively small dataset size, gamma=2 provided a better balance between hard example mining and stable learning.

experiment 1 loss example from drop=0.7 and fold 5:
`Train Loss: 0.0209 | Val Loss: 0.0357`
experiment 2 loss example from drop=0.7 and fold 5:
`Train Loss: 0.0035 | Val Loss: 0.0052`

Gamma=5 produces training loss ~10x lower (0.003 vs 0.02) but worse downstream metrics, this is the key finding. It shows the loss function became decoupled from actual segmentation quality. That's a research-level insight worth stating explicitly.

Even though focal loss became numerically smaller, segmentation quality decreased. High gamma overly emphasizes hard pixels and may destabilize learning and reduce generalization in moderately imbalanced datasets

This visualization from drop 0.7, fold 4 in Gamma = 2 and Gamma = 5



Gamma = 2 provides better balance between precision and recall, while Gamma = 5 reduces recall significantly. And moderate focal focusing ($\gamma = 2$) is more suitable for flood segmentation in this dataset.

These results suggest that optimizing focal loss value alone is not a reliable proxy for segmentation quality, and that downstream spatial metrics (IoU, Dice) must be monitored independently during training